

Roteiro do Vídeo

Apresentação da Galeria de Artistas Curitibanos

Projeto UIKit Avançado

Eduardo, João 1, Roberto e João 2

Aplicativo iOS desenvolvido com UIKit, View Code, UICollectionView, busca, detalhes, animações e compartilhamento.

Projeto: Galeria de Artistas Curitibanos

Duração recomendada: 12 a 15 minutos

Integrantes: Eduardo, João 1, Roberto e João 2

Observação: trocar "João 1" e "João 2" pelos nomes completos, se necessário.

Divisão geral

- Eduardo: abertura, proposta do app e demonstração inicial.
- João 1: arquitetura, modelo de dados e Assets Catalog.
- Roberto: UICollectionView, DataSource, Delegate e layout responsivo.
- João 2: busca, tela de detalhes, compartilhamento e animações.

1. Abertura - Eduardo - 1 a 2 minutos

Fala sugerida:

"Olá, professor. Nós somos o grupo formado por Eduardo, João, Roberto e João. Nosso projeto se chama Galeria de Artistas Curitibanos. A proposta do aplicativo é apresentar uma galeria interativa de artistas e obras ligadas a Curitiba, valorizando produções de artes visuais, arte urbana, música, teatro e televisão."

"O app foi desenvolvido em UIKit, usando View Code. Na tela inicial, o usuário encontra uma grade com imagens, títulos e nomes dos artistas. Ao tocar em um item, ele acessa uma tela de detalhes com informações completas e opção de compartilhamento."

Demonstração:

- Abrir o app no simulador ou device.
- Mostrar o cabeçalho da galeria.
- Rolar rapidamente a coleção.
- Destacar que existem obras de Poty Lazzarotto, João Turin e perfis de outros artistas curitibanos.

2. Arquitetura e dados - João 1 - 2 a 3 minutos

Fala sugerida:

"A arquitetura usada no projeto foi MVC, que é bastante comum em UIKit. Separamos o projeto em Model, View e Controller."

"No Model, temos a struct ObraDeArte, que representa cada item da galeria. Ela possui título, artista, ano, estilo, nome da imagem e descrição, exatamente como pedido no enunciado."

"Também temos o `CatalogoDeObras`, que centraliza a lista com todos os itens exibidos no app. Isso facilita a manutenção, porque a `CollectionView` lê os dados a partir dessa lista."

Mostrar no Xcode:

- `Models/ObraDeArte.swift`
- `struct ObraDeArte`
- `array CatalogoDeObras.obras`
- exemplos de itens como Poty Lazzarotto, João Turin e Karol Conka

Fala sobre assets:

"As imagens foram adicionadas localmente no Assets Catalog. Cada item possui um `imagemNome`, e esse nome corresponde a um `imageset` dentro do `Assets.xcassets`. Assim, o app consegue carregar as imagens com `UIImage(named: )`."

Mostrar:

- `Assets.xcassets`
- alguns `imagesets`: `poty_largo_da_ordem`, `turin_pieta`, `artista_karol_conka`

3. CollectionView, DataSource, Delegate e layout - Roberto - 3 a 4 minutos

Fala sugerida:

"A tela principal é controlada pela `GaleriaViewController`. Nela criamos uma `UICollectionView` programaticamente, usando View Code."

"A `CollectionView` usa uma célula customizada chamada `ObraCollectionViewCell`. Cada célula mostra a imagem, o título e o artista. Também aplicamos sombra, cantos arredondados, gradiente e uma linha laranja como detalhe visual."

Mostrar no Xcode:

- `Controllers/GaleriaViewController.swift`
- criação da `collectionView`
- `collectionView.dataSource = self`
- `collectionView.delegate = self`
- `Views/ObraCollectionViewCell.swift`

Fala sobre DataSource:

"No DataSource, o método `numberOfItemsInSection` retorna a quantidade de obras filtradas. O método `cellForItemAt` cria ou reutiliza a célula e chama `configurar(com:)`, passando a obra

correspondente."

Fala sobre Delegate:

"No Delegate, usamos `didSelectItemAt` para detectar o toque do usuário. Quando uma célula é tocada, pegamos a obra selecionada e abrimos a tela de detalhes."

Fala sobre layout:

"O layout responsivo foi feito com `UICollectionViewDelegateFlowLayout`. O tamanho das células é calculado com base na largura da tela, permitindo que a grade se adapte melhor em iPhone e iPad."

Demonstração:

- Girar o simulador ou comentar a adaptação.
- Mostrar a grade rolando.
- Tocar em uma célula para abrir detalhes.

4. Busca - João 2 - 2 minutos

Fala sugerida:

"A busca foi implementada com `UISearchController`. O usuário pode pesquisar por título, artista ou estilo."

"Um detalhe importante é que normalizamos o texto usando `folding(options:)`, então a busca ignora diferença entre maiúsculas, minúsculas e acentos. Isso melhora a experiência, porque uma busca por 'joao' também encontra 'João'."

Mostrar no Xcode:

- `UISearchResultsUpdating`
- método `updateSearchResults`
- função `normalizarBusca`

Demonstração:

- Pesquisar "poty".
- Pesquisar "joao".
- Pesquisar "rap" ou "teatro".
- Mostrar o estado vazio com uma busca sem resultado.

5. Tela de detalhes e compartilhamento - João 2 - 2 a 3 minutos

Fala sugerida:

"A tela de detalhes é controlada pela `DetalheViewController`. Ela recebe uma `ObraDeArte` no inicializador, então a tela já abre com os dados corretos do item selecionado."

"Nessa tela mostramos a imagem em destaque, título, artista, ano, estilo e descrição. Também usamos uma `ScrollView` para garantir que o conteúdo fique acessível em telas menores."

Mostrar no Xcode:

- `DetalheViewController`
- `init(obra:)`
- `popularDados()`
- `criarMetaItem`

Fala sobre compartilhamento:

"O botão Compartilhar usa `UIActivityViewController`. Ele monta uma mensagem com o título da obra, o artista e um convite para conhecer mais artistas curitibanos."

Demonstração:

- Abrir uma obra.
- Rolar a tela de detalhe.
- Tocar em Compartilhar.
- Mostrar a sheet de compartilhamento e cancelar.

6. Animações e UX - Eduardo - 1 a 2 minutos

Fala sugerida:

"Além dos requisitos principais, adicionamos animações sutis para melhorar a experiência do usuário."

"Na galeria, as células aparecem com uma leve animação de entrada. Ao tocar, a célula reduz um pouco de escala, dando feedback visual. Na tela de detalhes, os textos e o botão entram com uma animação em sequência."

"Também criamos um `DesignSystem` para centralizar cores, fontes, espaçamentos e raios de borda. A paleta verde e laranja foi escolhida para criar uma identidade visual marcante e ligada à ideia de cidade, cultura e arte urbana."

Mostrar:

- `Utils/DesignSystem.swift`
- animação em `ObraCollectionViewCell`
- animação em `DetalheViewController`

7. Dificuldades e soluções - Roberto - 1 a 2 minutos

Fala sugerida:

"Uma das dificuldades foi a configuração inicial do projeto. O app estava tentando abrir um storyboard chamado Main, mas o projeto foi feito em View Code. A solução foi remover a referência ao storyboard principal e criar a interface programaticamente."

"Outra dificuldade foi organizar as imagens locais. Como o requisito pedia imagens no Assets Catalog, conferimos se cada `imagemNome` tinha uma imagem correspondente."

"Também ajustamos a busca para lidar melhor com acentos e criamos um layout responsivo para a grade funcionar bem em diferentes tamanhos de tela."

8. Encerramento - todos - 1 minuto

Eduardo:

"Com isso, o app atende aos requisitos do projeto: usa UIKit, CollectionView, DataSource, Delegate, tela de detalhes, busca, animações e compartilhamento."

João 1:

"A estrutura MVC ajudou a manter o código organizado e fácil de explicar."

Roberto:

"A CollectionView foi o ponto principal do app, permitindo exibir a galeria de forma visual e interativa."

João 2:

"E a busca, os detalhes e o compartilhamento tornam a experiência mais completa para o usuário."

Todos:

"Obrigado!"

Checklist antes de gravar

- Rodar o app no simulador ou device.

- Testar busca por "poty", "joao", "rap" e "teatro".
- Abrir uma obra do Poty.
- Abrir um perfil de artista.
- Testar o botão Compartilhar.
- Mostrar rapidamente os arquivos principais no Xcode.
- Garantir que todos os integrantes falem.
- Manter o vídeo entre 10 e 20 minutos.